MCP Chat Backend Documentation

MCP Chat Backend — Project Documentation

Domain: Facilities / Building Inspection & Indoor Air Quality (IAQ) **Base URL (QA):** `https://qa-ai.sclera.com/sclera-mcp-chat-agent-server`

---

1. What This Project Is

An async, multi-tenant chat backend that sits between a React chat UI and an LLM agent (LangGraph/LangChain). The agent doesn't just answer in text — it calls tools over the **Model Context Protocol (MCP)** to fetch real data (inspections, buildings/IOCs, VDMS entities) and replies using a **strict UI-component JSON contract** so the frontend can render lists, detail cards, tables, and charts instead of plain text.

The README in the repo is stale (describes a sync Flask/SQLAlchemy app). The real, running system is **Quart (async) + LangGraph + async SQLAlchemy**, deployed via **Hypercorn**.

---

2. Architecture — Root to Branch

```
run.py                          ENTRY POINT — loads env, builds app, serves via Hypercorn on 0.0.0.0:8000
  │
  ▼
app/__init__.py                 create_app() — Quart application factory
  │  • quart_cors (CORS *)
  │  • registers 7 blueprints (chat, session, organization, mcp_models, org_mcp_config, tools, report/static)
  │  • before_serving → global_resources.initialize() + report cleanup thread
  │  • after_serving  → cleanup + engine.dispose()
  │
  ├─ app/config.py               Env-aware config (dev/qa/uat/staging/prod) + ~500-line UI_INSTRUCTIONS prompt
contract
  │     └─ AWS Secrets Manager → Postgres / OpenAI / JLL creds (env-var fallback, never crashes on missing secret)
  │
  ├─ app/extensions.py           Async SQLAlchemy engine + Base (legacy sync Flask-SQLAlchemy `db` kept for tests
only)
  │
  ├─ app/core/                   Startup singletons
  │     ├─ resources.py              ResourceManager: LLM instances, embeddings, Postgres pool,
  │     │                            LangGraph AsyncPostgresSaver checkpointer
  │     ├─ jll_token_manager.py      Okta M2M token fetch + background refresh (~every 50 min)
  │     ├─ mcp_session_cache.py      MultiServerMCPClient cache + reaper (expires idle MCP sessions)
  │     └─ mcp_bearer_interceptor.py contextvars-based interceptor: injects bearer token, ioc_id,
  │                                  timezone, loggedInUser into every outgoing MCP tool call
  │
  ├─ app/controllers/            HTTP routes (Quart blueprints): chat · session · organization ·
  │                              mcp_models · org_mcp_config · tools · report · static
  │
  ├─ app/services/               Business logic
  │     ├─ chat_service.py           THE CORE (~2000 lines): LangGraph agent build, SSE streaming,
  │     │                            MCP tool orchestration, UI-component tag parser, history
  │     │                            trimming/summarization
  │     ├─ session_service.py        Session CRUD, soft delete, rename
  │     ├─ report_service.py         Excel/PDF generation (reportlab, openpyxl) → S3 presigned URLs
  │     └─ pdf_export_service.py     Message → PDF export
  │
  ├─ app/repositories/           Async data access layer (SQLAlchemy select/insert/update):
  │                              chat · organization · mcp_model · org_mcp_config
  │
  ├─ app/utils/                  auth (PyJWT + JWKS + BFF validation) · aws_secrets · excel_importer ·
  │                              mcp_client · parsers · env_loader · db_setup (asyncpg) · callbacks ·
  │                              swagger_docs
  │
```

```
  └ app/models.py              SQLAlchemy tables: organizations, mcp_models, org_mcp_config,
                                sessions, chats

migrations/   Alembic (baseline, useremail, active_module, active_vdm_ids)
tests/        pytest + pytest-asyncio (still reference legacy Flask g / Flask-SQLAlchemy — stale)
Dockerfile    python:3.11-slim, 2-stage build, non-root user, healthcheck on :8000
```

---

3. Technology Stack

Layer	Technology	Notes
Language / runtime	Python 3.11, asyncio	
ASGI server	**Hypercorn**	what actually runs in `run.py` / Docker (uvicorn is listed in requirements but unused)
Web framework	**Quart** + quart-cors	async Flask-like; Flask/Flask-SQLAlchemy remnants exist only for legacy tests
Agent orchestration	**LangGraph** (`StateGraph` , `create_agent` , `AsyncPostgresSaver` checkpointer) + **LangChain** core	per-session memory persisted in Postgres
LLM provider	`langchain-openai` → OpenAI **or** JLL GPT (OpenAI-compatible, via Okta M2M + Azure APIM)	selected per-org / per-request via `model` field (`default` / `jll_gpt`)
Tool protocol	**MCP** via `langchain-mcp-adapters` (`MultiServerMCPClient`)	see §7 for MCP vs FastMCP
ORM / DB	SQLAlchemy 2.0 async (`create_async_engine`)	
Postgres drivers	`asyncpg` (app engine) + `psycopg` 3 / psycopg-pool (LangGraph checkpointer)	two drivers, two purposes
Migrations	Alembic	
Auth	PyJWT (JWKS validation) + BFF tenant re-validation	
Secrets / cloud	boto3 → AWS Secrets Manager; S3 for report storage (presigned URLs)	
Reports / files	reportlab (PDF), openpyxl + pandas (Excel import/export)	
API docs	Flasgger (Swagger via `swag_from`)	plus a hand-maintained Bruno/OpenAPI collection (`MCP_chat_backend.yaml`)
Token counting	tiktoken	used for history trimming/cost tracking

Vestigial / not active: Flask & Flask-SQLAlchemy (legacy test imports only), uvicorn, psycopg2-binary — all present in `requirements.txt` but not on the live code path.

---

4. Feature List

1. **Multi-tenant chat** — SSE-streamed conversational agent scoped to an organization (`org_id`) and, where relevant, a building/IOC (`ioc_id`) or up to 3 VDMS entities.
2. **Tool-using LLM agent** — LangGraph agent that autonomously chooses between local UI-formatting tools and remote MCP tools per turn.
3. **Structured UI responses** — enforced `<ui_component>` JSON contract (list, generic_details, table/chart, suggestion_chips, out_of_scope) so the chat UI renders rich widgets, not raw text.
4. **Session management** — create/list/rename/soft-delete sessions per logged-in user; each session keeps its own LangGraph checkpoint (memory).
5. **Chat history** — parsed, replayable message history per session (`/history/{sessionId}`).
6. **Organization management** — CRUD + bulk Excel import for tenant organizations.

7. **MCP model registry** — CRUD for registered LLM models (`provider`, `model_name`, `version`, `config`), with lookup by provider (e.g. `anthropic`).
8. **Org ↔ MCP config** — CRUD + Excel import linking an organization to specific MCP models/servers and API keys; supports enabling/disabling a config (`is_active`).
9. **Tool discovery** — `/tools` endpoint lists local tools plus live MCP-server tools, filterable by `org_id`, `property_id`, `base_url`, `server_url`, `server_name`, `transport` (e.g. `streamable_http`).
10. **PDF export** — export a rendered chat message (its parsed UI blocks) to a downloadable PDF.
11. **Per-org LLM routing** — dynamic OpenAI vs JLL GPT selection with Okta-token-based auth for JLL.
12. **History trimming/summarization** — keeps token usage bounded on long conversations (tiktoken-based).
13. **Health check** — unauthenticated liveness endpoint for container orchestration.
14. *(Disabled/planned)* File-download/report export in the UI contract — flagged "coming soon"; backend scaffolding exists but is commented out in `chat_service.py`.

---

5. Database Structure

Engine: PostgreSQL, accessed async via SQLAlchemy 2.0 (`asyncpg`) for app tables and via `psycopg` for the LangGraph checkpointer's own internal tables. Alembic manages schema migrations (`baseline`, `useremail`, `active_module`, `active_vdm_ids` revisions seen in history).

Core tables (`app/models.py`)

Table	Purpose	Key relationships
`organizations`	Tenant record (name, slug, status)	parent of `org_mcp_config`, `sessions`
`mcp_models`	Registered LLM/MCP model definitions (provider, model_name, version, config JSON)	referenced by `org_mcp_config`
`org_mcp_config`	Links an organization → an `mcp_models` entry + API-key reference + `is_active` flag	FK → `organizations`, FK → `mcp_models`
`sessions`	Per-user chat session (session_id, logged-in user, session_name, client_type, soft-delete flag)	parent of `chats`
`chats`	Message log — role, message text/blocks, token counts, cost, UI-component metadata	FK → `sessions`

```
organizations ──1:N── org_mcp_config ──N:1── mcp_models
      │
      └──1:N── sessions ──1:N── chats
```

Additional Postgres schema (managed separately by LangGraph, not Alembic): checkpoint tables created by `AsyncPostgresSaver` to persist per-session agent state/memory.

Table creation policy: tables auto-create on app startup only in `dev`; `qa`/`uat`/`staging`/`prod` rely exclusively on Alembic migrations.

---

6. API Surface (from the OpenAPI/Bruno collection)

Group	Endpoints
Chat	`POST /chat` (SSE stream — `session_id`, `prompt`

	required; `ioc_id` required unless `isVdms=true`; `model` = `default` | `jll_gpt`; `vdms` = up to 3 normalized VDMS ids), `GET /history/{sessionId}`
Sessions	`POST /sessions?loggedInUser=`, `GET /sessions?loggedInUser=`, `DELETE /sessions/{sessionId}`, `PATCH /sessions/{sessionId}/rename?loggedInUser=`
Organizations	`POST/GET /organizations/`, `GET/PUT/DELETE /organizations/{orgId}`, `POST /organizations/import/excel`
MCP Models	`POST/GET /mcp-models/`, `GET/PUT/DELETE /mcp-models/{modelId}`, `GET /mcp-models/provider/anthropic`
Org MCP Config	`POST/GET /org-mcp-config/`, `GET/PUT/DELETE /org-mcp-config/{configId}`, `GET /org-mcp-config/org/{orgId}`, `POST /org-mcp-config/import/excel`
Tools	`GET /tools/?org_id=` (lists local + live MCP tools; optional `property_id`, `base_url`, `server_url`, `server_name`, `transport`)
Report	`POST /export-message-pdf` (returns `application/pdf` attachment from already-parsed frontend message blocks)
Health	unauthenticated liveness check (no auth required — the only exception to the JWT rule)

All request bodies use `application/json` except Excel imports, which use `multipart/form-data` with a `file` field.

---

7. User / Client Access & Auth Flow

1. Every route except health requires a **JWT bearer token**.
2. The token's claims supply `org_id`. If the client also sends `org_id` in the request body, the backend **re-validates it against a BFF (backend-for-frontend)** rather than trusting the body blindly — prevents tenant spoofing.
3. `loggedInUser` is required as a query param on session endpoints (also accepted as `logged_in_user`), scoping sessions to the calling human user within their org.
4. Per-org LLM/API-key resolution happens via `org_mcp_config` + `org_config_manager.py` — so two orgs on the same deployment can use different models/keys.
5. For chat requests, a **contextvars-based bearer interceptor** (`mcp_bearer_interceptor.py`) transparently attaches the caller's token, `ioc_id`, timezone, and `loggedInUser` onto every outbound MCP tool call — so downstream MCP/FastMCP servers see the same identity/tenant context as the original request without the agent code having to thread it through manually.
6. `isVdms` / `vdms` params let a request target up to 3 specific VDMS entities instead of a single `ioc_id`; ids are normalized (`VDMS827`, `vdms 827`, `827` → `827`).

---

8. MCP vs FastMCP — What They Are and Why Both Exist

This system talks to **two different kinds of tool servers**, and the diagrammed architecture explains exactly why:

MCP Server (Spring AI) — the "native" path

- Used where the backend service is a **modern Spring Boot service** with **Spring AI** support.
- Tools are defined directly in the service code with a `@Tool` annotation, e.g. `@Tool getListOfInspections(description="gives`

`list of inspections")`, wrapping an already-existing service function.
- The LLM agent calls these tools, gets back the **raw JSON** the service function would normally return (e.g. a list of inspections), and the agent then maps that raw data into one of the predefined **local UI-JSON templates** (list, detail, table, chart) before returning it to the chat UI.

FastMCP Servers — the workaround for legacy services

- Some backend services (IOC backend, "boxes") run on an **older Spring Boot version that does not support Spring AI / `@Tool` annotations**.
- To still expose them as MCP tools without a framework upgrade, the project stands up a **lightweight FastMCP (Python) server** per legacy service.
- FastMCP is pointed at the service's **base URL + its OpenAPI spec**, and automatically generates one MCP `@tool` per selected endpoint — in this codebase, specifically the endpoints already namespaced under **`/mcp/*`** on that backend, so the legacy team only has to add plain REST endpoints (no annotations) and FastMCP does the MCP wrapping.
- Functionally, a FastMCP-wrapped `/mcp/*` endpoint behaves identically to a native `@Tool` from the agent's point of view — same raw-JSON-in, UI-JSON-out flow — the only difference is *how* the tool got exposed.

Why both are needed (in one line)

MCP-via-Spring-AI is used wherever the underlying service can support it; FastMCP is the bridge used only where the service's Spring Boot version is too old for Spring AI — both ultimately present themselves identically to the LangGraph agent and the `MultiServerMCPClient`.

End-to-end tool flow (both paths converge)

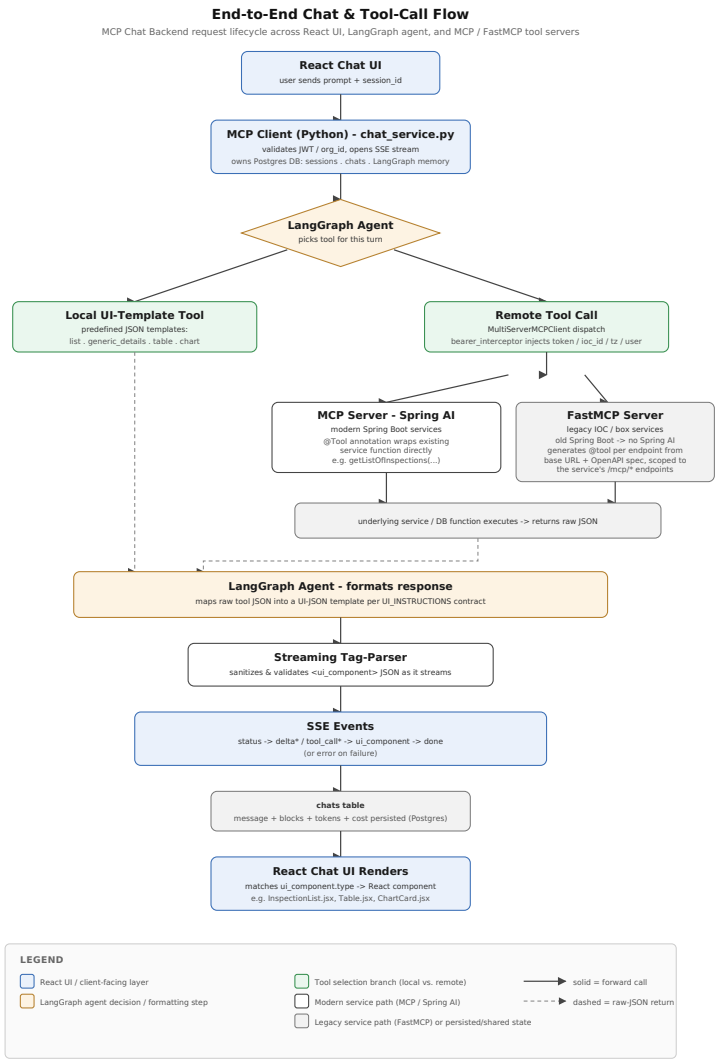
```
React Chat UI → prompt + context → MCP Client (Python, this repo)
   → LLM decides which tool to use (local UI-template tool OR remote MCP tool)
   → if remote: MultiServerMCPClient calls the MCP Server
        (Spring AI @Tool  – modern services)
        (FastMCP wrapper  – legacy Spring Boot IOC/box services, via their /mcp/* endpoints)
   → tool executes the underlying service function → returns raw JSON
   → agent maps raw JSON into a local UI-JSON template (list/detail/table/chart)
   → UI-JSON response streamed back over SSE
   → React renders it via the matching component (e.g. InspectionList.jsx, Table.jsx)
```

The MCP Client side also owns its **own Postgres database** (chats, sessions, LangChain/LangGraph memory) — separate from whatever database the MCP/FastMCP-fronted services use.

---

9. Flow Diagram — End-to-End Chat & Tool Call

The diagram below ties together everything in §7 and §8: how a single chat turn moves from the React UI down through the LangGraph agent, out to either a local UI-template tool or a remote MCP/FastMCP tool server, and back up as a rendered UI component.



End-to-end chat and tool-call flow

---

10. Documented Workflows

10.1 Chat turn (primary workflow)

1. Client `POST /chat` with `session_id`, `prompt`, `ioc_id` (or `vdms` list), `model`.
2. Backend validates JWT, re-validates `org_id` via BFF if supplied in body.
3. `chat_service.process_chat()` starts an async generator SSE stream.
4. **Fast-path short-circuits** run first: greetings, out-of-scope detection, known keyword triggers — answered without invoking the LLM.
5. If not short-circuited: LangGraph agent graph (`_build_graph`) is invoked with the Postgres-backed checkpointer for that `session_id` (restores prior turns/memory).
6. History is trimmed/summarized (tiktoken) to control context size.
7. LLM decides whether to call a local UI-template tool or a remote MCP/FastMCP tool.
8. If remote: `mcp_bearer_interceptor` injects token/`ioc_id`/timezone/`loggedInUser`; `MultiServerMCPClient`

dispatches to the right MCP or FastMCP server.
9. Tool result (raw JSON) returns to the agent, which formats it per `UI_INSTRUCTIONS` into a `<ui_component>` block (list/generic_details/chart/suggestion_chips/out_of_scope), preceded by exactly one lead sentence.
10. A streaming tag-parser sanitizes/validates the component JSON as it streams.
11. SSE events emitted in order: `status → delta*/tool_call* → ui_component → done` (or `error` on failure).
12. Chat message (with tokens/cost/blocks) is persisted to the `chats` table.

10.2 Session lifecycle

- **Create:** `POST /sessions?loggedInUser=` → server generates `session_id` (UUID), returns 201.
- **List:** `GET /sessions?loggedInUser=` — required user filter.
- **Rename:** `PATCH /sessions/{sessionId}/rename?loggedInUser=` — `session_name` ≤ 256 chars.
- **Delete:** `DELETE /sessions/{sessionId}` — soft delete, returns 204.
- **History replay:** `GET /history/{sessionId}` — returns parsed message history for re-rendering in the UI.

10.3 Organization onboarding

- `POST /organizations/` (or bulk `POST /organizations/import/excel`) creates tenant(s) with `name` + `slug` (+ `status`).
- `POST /org-mcp-config/` (or bulk `.../import/excel`) links that org to one or more `mcp_models`, tying in the API key reference and `is_active` flag — this is what makes an org's chat requests route to a specific LLM/tool set.
- `GET /org-mcp-config/org/{orgId}` used to inspect an org's active tool/model configuration.

10.4 MCP model registry management

- `POST /mcp-models/` registers a new model (`provider`, `model_name`, `version`, `config` e.g. `temperature`).
- `GET /mcp-models/provider/anthropic` filters registry by provider.
- `PUT/DELETE /mcp-models/{modelId}` updates/removes a registration.

10.5 Tool discovery

- `GET /tools/?org_id=` merges the local UI-template tool list with a live query of MCP-server tools reachable for that org (optionally scoped further by `property_id`, `base_url`, `server_url`, `server_name`, `transport`), useful for debugging what an org's agent can actually call.

10.6 PDF export

- Client sends already-parsed frontend message `blocks` (type + content) plus a `title` to `POST /export-message-pdf`; backend renders and returns an `application/pdf` attachment via reportlab. (The broader "download report to S3" flow exists in `report_service.py` but is currently disabled in the UI contract as "coming soon", with related code commented out in `chat_service.py`.)

---

11. Deployment

- **Container:** `Dockerfile` — 2-stage build on `python:3.11-slim`, runs as a **non-root user**, includes a container healthcheck against the app's health endpoint on port 8000.
- **Process:** `run.py` is the entrypoint; it loads environment config, builds the Quart app via `create_app()`, and serves it with **Hypercorn** (ASGI) bound to `0.0.0.0:8000`.
- **Windows dev note:** the entrypoint forces `SelectorEventLoop` on Windows because `psycopg` requires it.
- **Environments:** `dev`, `qa`, `uat`, `staging`, `prod`, selected via `.env.*` files loaded by `app/config.py`.
- **Secrets:** on startup, config attempts AWS Secrets Manager lookups for Postgres/OpenAI/JLL credentials, falling back to plain environment variables, and is designed not to crash the app if a secret is temporarily unavailable.
- **Schema management:** Alembic migrations are the source of truth in every environment except `dev`, where tables auto-create for convenience.
- **Observed live environment:** QA is hosted at `https://qa-ai.sclera.com/sclera-mcp-chat-agent-server`.
- **API documentation in the repo:** Flasgger/Swagger annotations in code, plus a separately maintained Bruno collection (exported as `MCP_chat_backend.yaml`) used for manual/API testing.

---

12. Known Tech Debt / Things to Watch

- **README is stale** — describes a synchronous Flask+SQLAlchemy app; the real system is async Quart+LangGraph. Don't trust it for onboarding.
- **Legacy Flask/Flask-SQLAlchemy code paths** remain only for old tests (`tests/` still import `flask.g`) — not part of the running app.
- **uvicorn and psycopg2-binary** are listed as dependencies but unused; Hypercorn + asyncpg/psycopg3 are what's actually active.
- **Report/file-download path** is implemented but deliberately disabled in the UI ("coming soon"), with dead/commented scaffolding in `chat_service.py` — don't assume it's live.
- **Most active development churn** is in pagination correctness and streaming/buffer edge cases inside `chat_service.py` — this is the file to check first for recent bugs or behavior changes.
- **Two Postgres roles co-exist in one database:** application tables (via SQLAlchemy/asyncpg) and LangGraph's own checkpoint tables (via psycopg) — worth knowing when reasoning about migrations, since only the former goes through Alembic.